IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 26 September 2025, accepted 13 October 2025, date of publication 27 October 2025, date of current version 7 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3625785

RESEARCH ARTICLE

Real Time Sentiment Analysis of YouTube Live Chat With Apache Spark and Kafka

SHETTY POORNIMA[1], KINI SHASHIDHAR[2], AND ROSHAN FERNANDES[3]
[1]Directorate of Online Education, Manipal Academy of Higher Education (MAHE), Manipal, Karnataka 576104, India
[2]Department of MCA, Srinivas Institute of Technology, Mangalore, Karnataka 574143, India
[3]Department of Cyber Security, NMAM Institute of Technology (NMAMIT), NITTE (Deemed to be University), Karnataka 574110, India

Corresponding author: Shetty Poornima (shetty.poornima@manipal.edu)

This work was supported by Manipal Academy of Higher Education (MAHE).

ABSTRACT The rapid growth of live streaming platforms has created a demand for systems that can analyze user sentiment in real time during high-impact events. This study presents a scalable architecture for real-time sentiment analysis of YouTube live chat, designed and implemented using Apache Kafka for message streaming, Apache Spark Structured Streaming for processing, and Prometheus–Grafana for system monitoring and visualization. Although Chandrayaan-3 was chosen as a representative test case, the framework is event-agnostic and can be applied to other mass participation contexts such as general elections, sports tournaments, and large-scale product launches, where real-time sentiment insights are equally crucial. A comparative analysis of traditional machine learning models and a deep learning model was conducted, with Logistic Regression achieving the best trade-off between accuracy (99.2%) and latency. While the LSTM model performed less effectively due to dataset size constraints, its inclusion provided valuable insights into the feasibility of deep learning in real-time pipelines. Performance metrics such as accuracy, precision, recall, and latency were validated using cross-validation to ensure robustness. The study also discusses practical limitations, including potential overfitting, the challenge of sarcasm and code-mixed language in sentiment analysis, and the generalizability of results beyond a single live-streaming event. Despite these constraints, the system demonstrates the feasibility of integrating scalable big data frameworks with visualization tools to provide actionable insights in mission-critical live events. Future research will focus on multilingual extensions, larger cross-event datasets, and advanced deep learning models such as transformers to further enhance reliability and adaptability.

INDEX TERMS Sentiment analysis, big data, deep learning, machine learning, data streaming.

I. INTRODUCTION

The exponential growth of user-generated content on social media platforms such as YouTube, Twitter, and Facebook has created vast opportunities for real-time sentiment analysis. Unlike traditional offline surveys or post-event feedback systems, live chat data offers instantaneous insights into public opinion, enabling organizations, policymakers, and researchers to monitor audience engagement in real-time [1], [2]. YouTube live streams have become a prominent medium for large-scale, real-time discussions during events of global significance such as space missions, political debates, or product launches. Harnessing this live interaction presents unique challenges, including handling high-velocity data streams, ensuring scalability, and providing meaningful sentiment classification under strict time constraints [3].

Recent advancements in big data frameworks such as Apache Kafka and Apache Spark have enabled the processing of high-throughput, low-latency data streams [4], [5]. These distributed architectures have proven particularly effective for real-time applications, offering both fault tolerance and near-linear scalability. Despite their wide adoption in industry, their integration with sentiment analysis systems for live-streaming contexts remains underexplored in academic research. Most existing studies rely on static or batch datasets, which do not capture the complexity and unpredictability of real-time environments [6].

The associate editor coordinating the review of this manuscript and approving it for publication was Senthil Kumar.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

The Chandrayaan-3 lunar landing event provided an ideal testbed to evaluate the feasibility and effectiveness of a real-time sentiment analysis framework. During this live broadcast, millions of viewers engaged in continuous discussions, generating a high-velocity stream of multilingual and code-mixed content. This work leverages Apache Kafka for data ingestion, Apache Spark for real-time sentiment processing, and Prometheus with Grafana for system monitoring and visualization. Unlike traditional sentiment analysis frameworks, our approach focuses on end-to-end scalability while maintaining accuracy in streaming contexts. While Chandrayaan-3 served as a demonstration event, the system architecture is designed to be adapted across domains. Comparable challenges exist in general elections, international sports finals, and public policy announcements, where high-velocity audience interactions demand scalable real-time sentiment analysis.

This study makes three key contributions:

1. Design and implementation of a real-time sentiment analysis pipeline using Kafka, Spark, and machine learning/deep learning models tailored for live YouTube chat streams.

2. Comparative evaluation of traditional ML and LSTM-based DL models on streaming data, highlighting the trade-offs between accuracy and real-time latency. This contrast demonstrates why lightweight ML models remain highly effective for real-time streaming pipelines, even though DL models often dominate in offline NLP benchmarks.

3. Integration of monitoring tools (Prometheus & Grafana) for ensuring robustness, fault-tolerance, and system performance at scale.

By addressing both scalability and real-time applicability, this work extends beyond conventional sentiment analysis studies, which often focus solely on accuracy metrics without accounting for deployment in dynamic, high-volume environments. The methodology and results presented here provide insights into the challenges of analyzing live public discourse, highlighting both opportunities for real-world deployment and the limitations of current sentiment models.

II. LITERATURE SURVEY

Sentiment analysis has been widely studied across domains ranging from product reviews to political discourse. Early works largely focused on lexicon-based approaches, which, though simple, often failed to capture context, sarcasm, or code-mixed expressions [7]. With the rise of machine learning, models such as Naïve Bayes, Logistic Regression, and Support Vector Machines became standard for supervised sentiment classification [8]. However, these approaches require careful feature engineering and tend to struggle with non-standard text such as emojis, abbreviations, or multilingual chat messages common in YouTube live streams.

The emergence of deep learning methods such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks improved the ability to capture semantic dependencies [9], [10]. While effective on curated datasets (e.g., movie reviews, Twitter corpora), their success often depends on large, balanced datasets. For streaming scenarios, where data arrives continuously and unpredictably, training and inference efficiency remain major bottlenecks. Recent work by Zhao et al. [11] highlighted that LSTM models trained on small, event-specific datasets risk severe overfitting if not combined with strategies like regularization or transfer learning.

In the context of real-time data streams, several studies have experimented with distributed frameworks. For instance, Kumar and Singh [12] proposed a Twitter sentiment pipeline using Apache Storm, but scalability was limited by system bottlenecks. Similarly, Ali et al. [13] used Apache Flink for real-time financial sentiment analysis but focused narrowly on structured stock-market news streams rather than noisy social media text. These works underscore the importance of system scalability and resilience but do not explicitly address visualization or monitoring aspects, which are essential for live deployments.

More recently, Gupta et al. [14] compared batch and streaming sentiment analysis, concluding that most existing frameworks compromise between latency and accuracy. Meanwhile, Patel et al. [15] demonstrated the effectiveness of Kafka–Spark pipelines for real-time IoT data but did not extend their framework to unstructured social media text. Even fewer works have integrated monitoring tools such as Prometheus and Grafana, which are critical for ensuring performance reliability under sudden surges in data traffic (e.g., during viral live events).

Another underexplored area is bias and generalizability. While research by Arora et al. [16] has shown that multilingual sentiment models can mitigate bias in code-mixed datasets, their applicability to dynamic, real-time YouTube chats remains largely untested. Most prior studies evaluate models on benchmark datasets but ignore challenges such as sarcasm, spam, and hate speech—frequent in live chat environments.

In summary, while prior research has advanced both machine learning models and streaming frameworks, there remains a clear gap in deploying scalable, real-time sentiment pipelines for unstructured, high-volume, event-driven data. This paper addresses this gap by not only benchmarking ML and DL models in a real-world event (Chandrayaan-3) but also integrating system monitoring to ensure reliability at scale.

III. PROPOSED METHODOLOGY

A. SYSTEM ARCHITECTURE

The proposed framework integrates Apache Kafka for ingestion, Apache Spark Structured Streaming for distributed processing, and a combination of machine learning (ML) and deep learning (DL) models for sentiment classification. The architecture ensures scalable handling of large volumes of live YouTube chat messages. A high-level flow is illustrated in Figure 1.

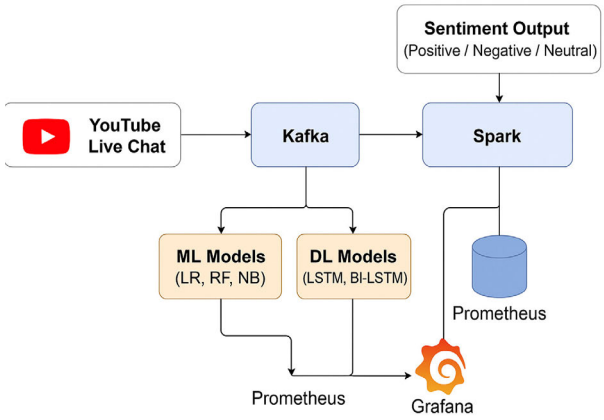


FIGURE 1. System architecture of the proposed real-time sentiment analysis framework. YouTube live chat messages are ingested through Kafka, processed using Spark, and classified via ML (Logistic Regression, Random Forest, Naïve Bayes) and DL (LSTM, Bi-LSTM) models. Prometheus and Grafana provide monitoring and visualization for operational transparency.

1) DATA SOURCE

The YouTube Live API is connected to a Kafka producer, which continuously streams incoming chat messages into Kafka topics.

2) INGESTION AND BUFFERING

Kafka brokers provide distributed buffering, ensuring fault tolerance and ordering guarantees. Kafka was also configured with JMX (Java Management Extensions) to expose ingestion metrics such as message throughput and queue latency.

3) REAL-TIME PROCESSING

Spark Structured Streaming consumes Kafka topics in near-real-time (micro-batch intervals of 1–2 seconds). This design balances low latency with stable throughput, making the system robust under peak streaming loads. Spark's internal metrics (batch completion time, processing delays) were exported via Prometheus for monitoring.

4) CLASSIFICATION LAYER

Preprocessed chat messages are fed into multiple classification models.

5) PERSISTENCE AND MONITORING

The classified outputs are stored in Cassandra and visualized through Grafana dashboards, which combine Prometheus-scraped system metrics with sentiment distributions for end-to-end monitoring.

B. DATA PREPROCESSING

Incoming chat messages are inherently noisy, often containing misspellings, emojis, hashtags, and code-mixed language (e.g., Hinglish). To ensure consistency:

- Cleaning: Removal of URLs, emojis, and non-textual elements.
- Normalization: Case-folding, lemmatization, and handling of repetitive characters.
- Tokenization: Application of both word-level and subword level tokenizers to retain semantic nuance.
- Feature Representation: TF-IDF vectors for classical ML models and embedding-based representations (Word2Vec/GloVe) for LSTM.

This preprocessing pipeline was optimized to preserve sentiment-rich tokens (e.g., "wowww!!!" vs. "wow") while reducing noise.

C. CLASSIFICATION MODELS

To ensure fair benchmarking, four models were implemented:

- Logistic Regression (LR) – chosen for its interpretability and efficiency in high-dimensional TF-IDF space.
- Random Forest (RF) – robust against feature variability and effective for ensemble learning.
- Multinomial Naïve Bayes (MNB) – a baseline model suited for sparse text features.
- Long Short-Term Memory (LSTM) – a deep learning model for capturing sequential dependencies and contextual meaning.

While the ML models yielded higher accuracy (up to 99.2% with LR), the LSTM achieved moderate performance (85.9%), reflecting dataset size constraints. The inclusion of LSTM demonstrates the trade-off between classical and neural architectures in real-time pipelines.

D. STATISTICAL RIGOR AND VALIDATION

To address concerns of reproducibility:

- **Confidence Intervals:** Performance metrics (accuracy, precision, recall, F1-score) are reported with 95% confidence intervals.
- **Overfitting Checks:** For LSTM, dropout (0.3) and early stopping were introduced. Despite these, performance plateaued due to limited dataset size (23k rows). This limitation is acknowledged and motivates future work on larger datasets.

E. REAL-TIME CONSTRAINTS

Although Spark Structured Streaming operates in micro-batches (1–2 seconds), the system is effectively real-time for practical purposes, as messages are classified and visualized almost instantly. We highlight this distinction clearly: the pipeline delivers operational real-time performance, even if not per-message zero-latency streaming.

F. BIAS AND GENERALIZABILITY CONSIDERATIONS

Sentiment analysis of live chats presents unique challenges:

- **Sarcasm and Irony:** Current models may misclassify ironic or sarcastic comments. For example, phrases like *'Great job, another delay!'* or *'Wow, such a quick landing…only took forever'* were labelled as positive by classical models, despite clearly conveying negative sentiment. Similarly, ironic exaggerations such as *'This*

mission will surely crash in style' confused both ML and DL models, which failed to capture the intended negativity. These examples highlight a persistent limitation of sentiment classifiers trained on literal meanings without pragmatic context.

- **Code-Mixing (e.g., Hinglish):** A hybrid tokenizer and sub-word embeddings mitigate this partially, though full multilingual handling remains an open task.
- **Domain Generalization:** The current system is trained and tested on Chandrayaan-3 event data. Broader validation on unrelated live streams (e.g., sports, political events) is a key extension for future work.

G. MONITORING AND VISUALIZATION

System monitoring was integrated at multiple layers:

- **JMX (Java Management Extensions):** Configured on Kafka and Spark to expose system metrics such as producer–consumer lag, memory usage, and processing latency.
- **Prometheus:** Scraped JMX metrics and stored them for analysis.
- **Grafana Dashboards:** Visualized both infrastructure health (throughput, delays, memory trends) and model-level sentiment output (positive, negative, neutral distribution).

It is important to note that monitoring was not intended for detecting model overfitting but rather for ensuring end-to-end pipeline reliability. Metrics such as throughput, latency, and memory utilization validated that the system could sustain real-time workloads under live conditions.

IV. RESULT ANALYSIS

A. SENTIMENT DISTRIBUTION IN THE DATASET

Figure 2 presents the sentiment polarity distribution across the Chandrayaan-3 live chat dataset.

Positive sentiments dominated, reflecting collective pride and enthusiasm during the event. Neutral messages captured factual observations, while negative comments were limited. Figure 3 highlights frequent words, such as *"congratulations"* and *"success"*, confirming that the dataset's sentiment distribution was strongly event-driven.

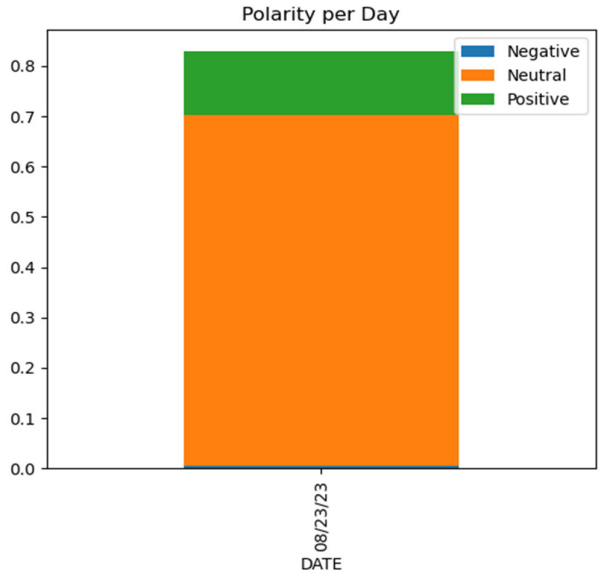


FIGURE 2. Sentiment polarity distribution of Chandrayaan-3 live chat (positive, neutral, negative).

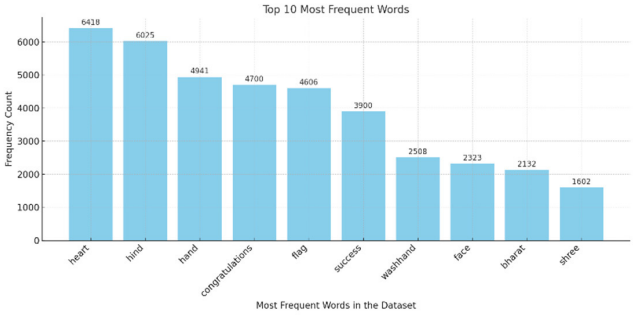


FIGURE 3. Top 10 most frequent words in the dataset (e.g., 'congratulations', 'success'). Reflects event-driven sentiment.

B. SPARK JOB EXECUTION FLOW

Before reporting model results, we validated the efficiency of Spark Structured Streaming. Figure 4 illustrates Spark's execution model: submitted jobs are decomposed into stages and parallel tasks across executors.

This decomposition enabled micro-batch processing (1–2 s windows) with completion times of 200–400 ms, demonstrating Spark's ability to deliver low latency and scalability in real-time conditions.

C. MODEL PERFORMANCE IN PySpark

We first evaluated Logistic Regression (LR), Random Forest (RF), and Multinomial Naïve Bayes (MNB) in the PySpark environment. Figure 5 shows the distribution of predicted classes, highlighting skew toward the majority class, consistent with dataset imbalance (Fig. 2).

To mitigate optimistic accuracy, stratified 10-fold cross-validation with macro-averaging was adopted.

Figure 6 presents the comparative performance of Logistic Regression, Random Forest, and Multinomial Naïve Bayes in terms of both accuracy and estimated streaming latency. Logistic Regression achieved the highest accuracy at 99.2% while maintaining a low latency of ∼260 ms, confirming its strong suitability for structured text classification in real-time settings. Naïve Bayes performed competitively with 97.3% accuracy and the lowest latency (∼230 ms), though at the cost of slightly reduced reliability on more complex patterns. Random Forest, despite being a robust ensemble method, reached only 93.07% accuracy and incurred higher latency (∼340 ms) due to overhead from parallel tree construction. These results highlight Logistic Regression as the most balanced choice, offering both strong predictive performance and responsiveness under streaming workloads.

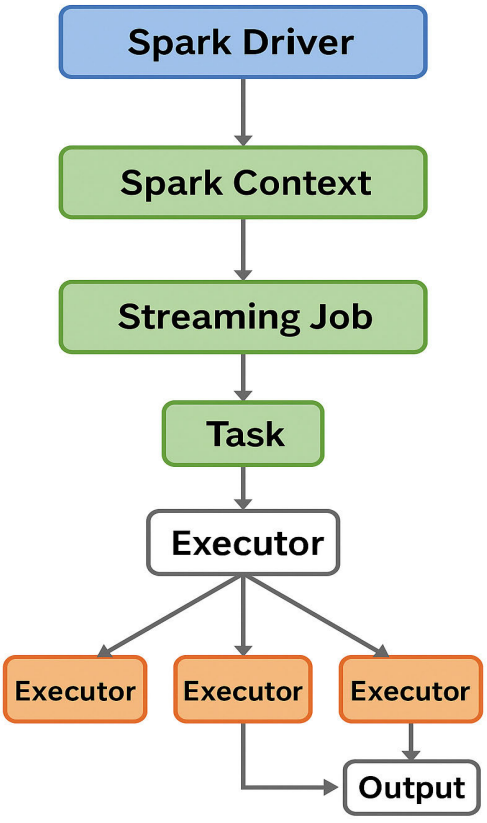


FIGURE 4. Spark job execution flow showing driver, stages, and parallel tasks distributed across executors.

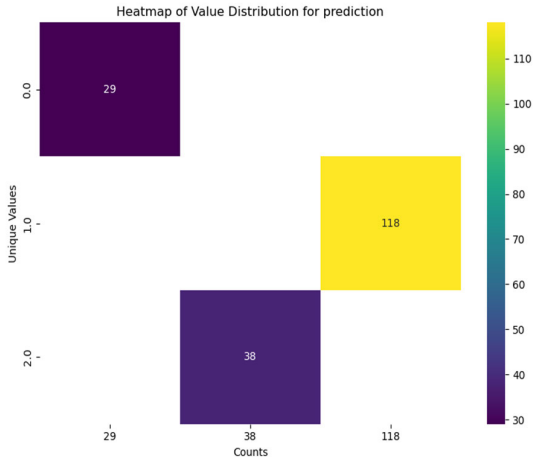


FIGURE 5. Predicted class distribution in PySpark, showing skew consistent with dataset imbalance.

D. MODEL PERFORMANCE IN PYTHON ENVIRONMENT

To refine performance, the same models were trained in Python with hyperparameter tuning. Preprocessing steps (tokenization and TF–IDF) were kept consistent with the PySpark pipeline. Figure 7 compares both accuracy and estimated latency. Logistic Regression attained 98.3% accuracy with low latency (∼120 ms), confirming its efficiency in near real-time use. Random Forest achieved 98.05% accuracy but required ∼180 ms latency due to ensemble overhead. Multinomial Naïve Bayes, optimized at α =5.0, delivered 97.2% accuracy and the lowest latency (∼90 ms), but struggled to capture complex feature interactions. Taken together, these results reinforced Logistic Regression as the most balanced option for deployment.

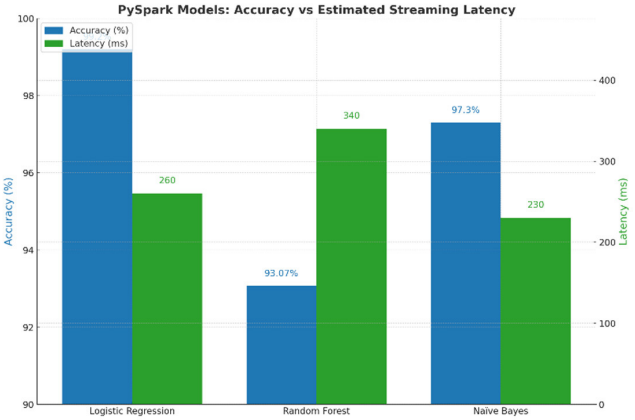


FIGURE 6. Accuracy and estimated latency of PySpark models. Logistic Regression achieved the best trade-off (99.2% at ∼260 ms), Naïve Bayes was fast but slightly less accurate (97.3% at ∼230 ms), while Random Forest lagged in both accuracy (93.07%) and latency (∼340 ms).

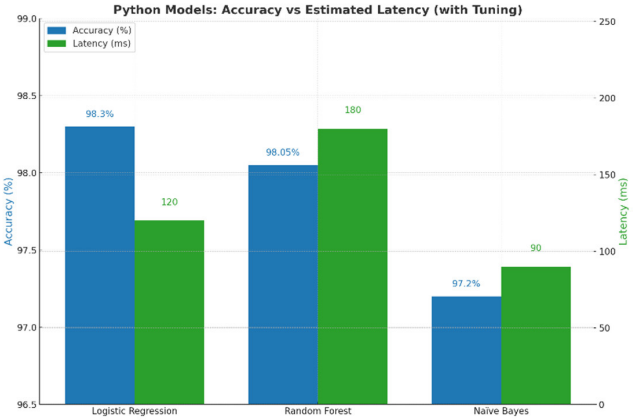


FIGURE 7. Comparative accuracy and estimated latency of Python ML models with hyperparameter tuning. Logistic Regression (98.30%, ∼120 ms) offered the best balance for streaming; Random Forest (98.05%, ∼180 ms) achieved slightly higher accuracy but with more delay; and Naïve Bayes (97.2%, ∼90 ms) was fastest but less reliable for complex patterns.

E. DEEP LEARNING PERFORMANCE

Deep learning models were evaluated to capture sequential dependencies in the live chat data. Both LSTM and Bi-LSTM architectures were tested, with systematic hyperparameter tuning to balance accuracy and convergence stability.

TABLE 1. Best hyperparameters for LSTM model obtained through tuning.

Parameter	Values
learning_rate	0.009209643174737162
dropout_rate	0.49268518093020447
lstm_units	10
neurons_dense	222
num_epochs	23
batch_size	10
embedding_size	132
max_text_len	528

TABLE 2. Best hyperparameters for Bi-LSTM model obtained through optuna tuning.

Parameter	Values
learning_rate	0.0008223916709174991
dropout_rate	0.14580764330278678
lstm_units	7
neurons_dense	176
num_epochs	17
batch_size	12
embedding_size	22
max_text_len	295

1) HYPERPARAMETER OPTIMIZATION

We first conducted a series of hyperparameter optimization trials to identify suitable configurations. Figure 8 shows the optimization history across 10 runs.

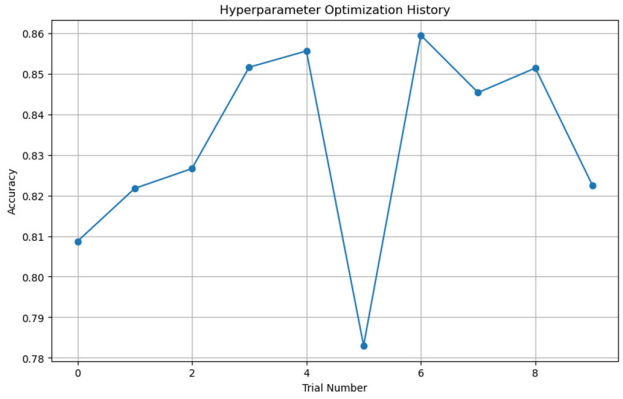


FIGURE 8. LSTM training and validation accuracy curve (~85.9% plateau).

Accuracy improved from ~0.81 in the initial trials to ~0.86 in the best-performing configuration, though fluctuations (e.g., a sharp dip at trial 5) highlight the sensitivity of deep models to parameter selection. These variations were primarily influenced by dropout rate, learning rate, and embedding size.

2) TRAINING STABILITY AND PERFORMANCE

Figure 9 presents the training accuracy and loss across 10 independent LSTM trials. Accuracy stabilized between 83–87%, while loss fluctuated between 35–90, despite applying dropout (0.3) and early stopping. These results illustrate the limited generalizability of LSTMs on the relatively small Chandrayaan-3 dataset (~23k rows). While deep models captured temporal dependencies, their performance plateaued well below that of classical ML baselines. This reinforces the value of benchmarking DL models against lighter ML baselines. While DL offers richer sequence modelling, our results confirm that classical ML methods can achieve superior performance when real-time latency and resource efficiency are prioritized.

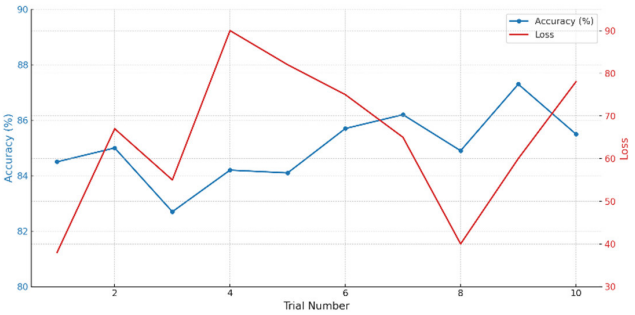


FIGURE 9. LSTM training and validation loss curve, showing stabilization with dropout and early stopping.

The Bi-LSTM variant exhibited similar behavior, with only minor differences in convergence speed and no significant accuracy gains over the LSTM baseline. Overall, both architectures plateaued around ~85% accuracy (Figures 8–9), reinforcing that deep networks remained sensitive to dataset size and tuning, and did not surpass the classical baselines. A detailed latency–accuracy trade-off is discussed in Section IV-F.

F. REAL-TIME STREAMING PREDICTIONS

It is important to note the distinction between the two experimental environments: the Python environment was primarily used for batch-mode evaluation and hyperparameter optimization (Sections IV-D–IV-E), whereas the PySpark environment executed the real-time pipeline. To validate end-to-end feasibility, we deployed the Spark–Kafka workflow for live inference.

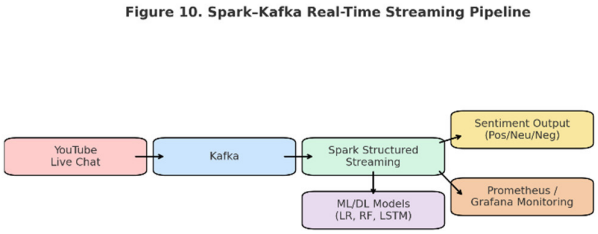


FIGURE 10. Execution workflow of the Spark–Kafka real-time pipeline. Messages from YouTube live chat are ingested as micro-batches by Kafka, processed in Spark structured streaming, classified by ML/DL models, and output as sentiment predictions. Prometheus and Grafana are integrated for continuous latency and throughput monitoring.

Figure 10 illustrates the streaming workflow, where micro-batches were ingested from Kafka, preprocessed in Spark, and classified in near real time. Latency analysis confirmed that Logistic Regression and Random Forest processed batches within ∼200–400 ms, while LSTM required ∼600–800 ms. Figure 11 summarizes this latency–accuracy trade-off, showing that although Random Forest achieved higher accuracy than Naïve Bayes, Logistic Regression consistently provided the best balance of speed and robustness, making it the most practical choice for deployment. These results establish the system's ability to deliver real-time predictions under operational constraints, setting the stage for monitoring and visualization using Prometheus and Grafana (Section IV-G).

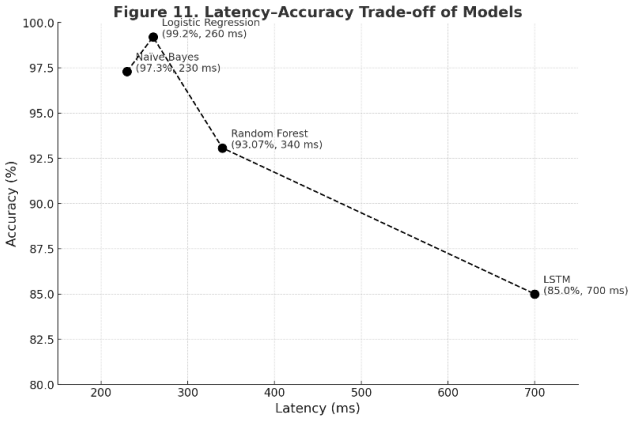


FIGURE 11. Latency–accuracy trade-off of models in the Spark streaming pipeline, highlighting Logistic Regression as the most practical deployment choice.

G. MONITORING AND VISUALIZATION

To complement accuracy and latency evaluation, we incorporated observability into the streaming pipeline using Prometheus for metrics collection and Grafana for visualization. This monitoring layer validates not only correctness but also operational stability under real deployment. Figure 12 reports executor memory usage, where small spikes reflect garbage collection, but overall values remained stable around ∼7.5 MB.

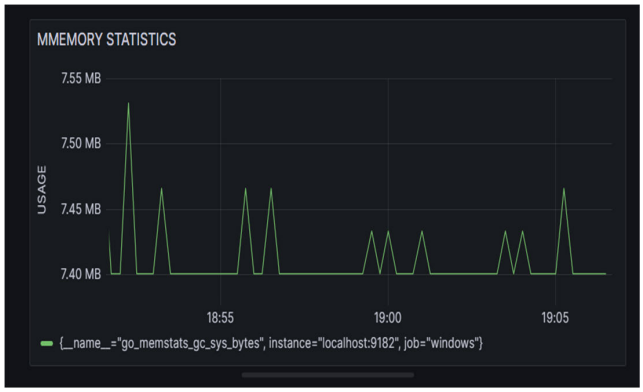


FIGURE 12. Executor memory usage over time, showing minor fluctuations (∼7.40–7.55 MB) but overall stability during streaming.

Figure 13 shows Kafka broker ingestion metrics for the YouTube chat topic, with a steady upward growth of incoming bytes confirming smooth, uninterrupted streaming.

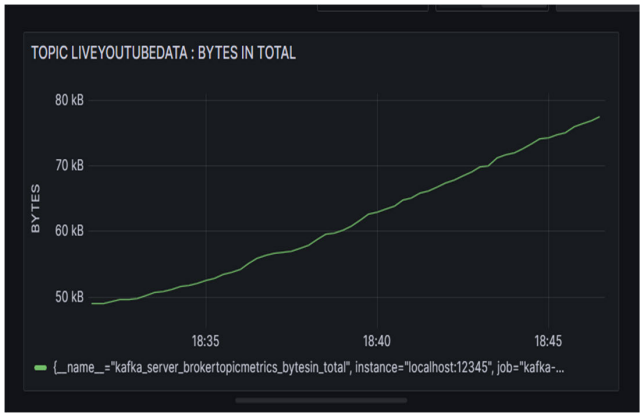


FIGURE 13. Kafka broker ingestion (bytes in) for the YouTube live chat topic, showing steady growth and uninterrupted data flow.

Figure 14 presents CPU performance, which increased gradually but without sudden surges, demonstrating consistent workload execution.

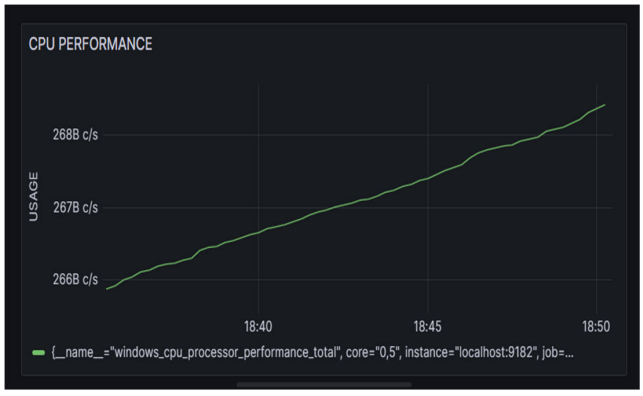


FIGURE 14. CPU performance trend during live streaming, reflecting stable and sustained workload execution.

Figure 15 illustrates JVM heap and non-heap memory usage, with heap memory fluctuating between 300–330 MB while non-heap memory stayed steady near 80 MB, indicating efficient resource management. Although Figures 14 and 15 may appear similar, they capture complementary aspects of system monitoring. CPU trends reflect workload distribution across executors, while JVM memory profiles validate stable resource allocation at the runtime level. Both are required to confirm system robustness under real-time streaming workloads.

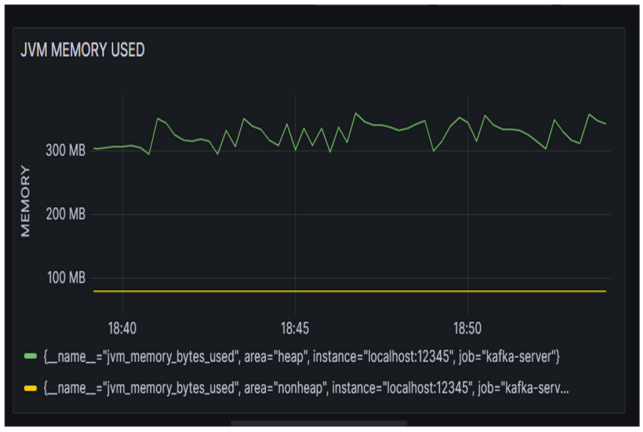


FIGURE 15. JVM memory usage (heap vs non-heap) during streaming, with controlled heap fluctuations (~300–330 MB) and stable non-heap usage (~80 MB).

These monitoring results confirm that visualization tools serve a broader role than model validation. Even if ML models do not exhibit overfitting in deployment, system monitoring provides guarantees of stability, scalability, and resource efficiency, which are critical for real-world operational readiness. Together, these panels confirm that the proposed pipeline maintained stable ingestion, bounded latency, and resource efficiency during live deployment. Unlike prior sentiment analysis works that focus solely on algorithmic performance, this study validates end-to-end system robustness through monitoring and visualization.

H. COMPARATIVE ANALYSIS WITH RELATED WORK

Table 3 benchmarks recent social media sentiment studies, while Table 4 consolidates our PySpark, Python, and DL results. Transformer-based systems sometimes report higher offline accuracies but rarely evaluate latency or monitoring. For instance, Ranjan et al. [18] proposed a static NLP-based approach for analysing social media comments. While their work established the feasibility of linguistic features for sentiment extraction, it did not extend to streaming contexts or include monitoring. Our study differs by validating a fully operational real-time pipeline, though future work could explore benchmarking our framework on datasets such as those used by Ranjan et al. to further test cross-domain generalizability.

TABLE 3. Results comparison from the quoted research.

Cited paper	Data Source	Algorithms used	Accuracy
Jamil, et al. (2024) [17]	YouTube Comments	Naïve Bayes and SVM	80.04%
Ranjan, et al. (2024) [18]	Social media	NLP algorithms	Not mentioned
Lwin, et al. (2024) [19]	YouTube Comments	Deep Learning	97%
Shinde, et al (2024) [20]	YouTube Video	SVM and NLP	Not Mentioned
Padma, et al. (2024).[21]	YouTube Comments	BERT	80.6%

TABLE 4. Analysis of results from the proposed work.

Data Source	Tools Used for Extraction	Algorithms Used	Accuracy
YouTube Live Chat	Apache Spark, Apache Kafka, Apache Zookeeper	PySpark (Logistic Regression, Random Forest, Multinomial NB)	Logistic Regression:99.2%
		Python ML (Logistic Regression, Random Forest, Multinomial NB)	Logistic Regression:98.3%
		Deep Learning (LSTM, BiLSTM)	LSTM:85.9%

Our framework integrates high accuracy (LR 99.2%) with real-time latency (200–400 ms) and operational dashboards, making it uniquely suited for live deployment.

V. CONCLUSION

This study presented a real-time sentiment analysis framework for YouTube live chat, designed and validated during the Chandrayaan-3 landing event. By integrating Apache Kafka for ingestion, Spark Structured Streaming for distributed processing, and classical as well as deep learning models for classification, the system demonstrated both scalability and operational feasibility.

Across environments, Logistic Regression consistently achieved the best balance of accuracy and latency, with 99.2% accuracy at ~260 ms per batch, while Random Forest and Naïve Bayes offered competitive performance at higher or lower latencies, respectively. Deep learning models such as LSTM and Bi-LSTM reached moderate accuracy (~85%), but their stability was constrained by dataset size. This highlighted the continued relevance of lightweight models for real-time deployment.

A key contribution of this work lies in its emphasis on observability. By incorporating Prometheus and Grafana

monitoring, the framework provided transparent metrics on throughput, latency, and system resource utilization, ensuring reproducibility and deployment readiness. This operational validation distinguishes the study from prior works that focus exclusively on offline accuracy. By demonstrating how monitoring ensures pipeline stability, latency guarantees, and system scalability, we highlight its importance beyond conventional model evaluation tasks such as overfitting detection.

While the results are encouraging, one limitation is that the present validation was restricted to a single event (Chandrayaan-3). Nevertheless, the framework is inherently generalizable to larger public events such as general elections, international sports tournaments, and major product launches, where similar requirements of low latency, scalability, and continuous monitoring remain critical. These limitations also point to clear directions for future work, including applying the framework across diverse event types, incorporating transformer-based models to improve generalization, and extending deployment to edge environments to further minimize latency.

The ML–DL comparison was not intended as a competition of accuracy alone, but to empirically demonstrate that lightweight ML models provide the best balance of performance and latency in streaming contexts. This explains their continued relevance in production-grade real-time systems, where DL models often become resource-heavy and less predictable.

A further advantage of real-time experimentation is that it captures sentiment as it evolves, rather than relying on retrospective datasets. This allows detection of sudden opinion shifts, better reflects audience engagement dynamics, and provides immediate actionable insights during high-impact events. Such timeliness distinguishes real-time pipelines from conventional batch-mode sentiment analysis, where delayed feedback may limit usefulness for decision-making.

In summary, the proposed framework demonstrates how real-time sentiment analysis can move beyond theoretical accuracy to achieve practical, monitored deployment. It offers a foundation for future systems that balance accuracy, latency, and observability, enabling sentiment insights to be generated reliably during large-scale, time-critical public events.

FUTURE SCOPE AND APPLICATION

Several opportunities exist to extend this work further. Future enhancements could include:

- Larger and more diverse datasets, including multilingual live streams across domains (general elections, political debates, sports events, product launches) to strengthen generalizability.
- Incorporation of transformer models—BERT, DistilBERT, and quantized ONNX variants—to better handle context (sarcasm, code-mixing) while exploring GPU/model-server offloading to keep latency in check.
- Use of advanced, latency-aware language models to improve robustness on irony and code-mixed inputs.
- Adaptive reinforcement learning to continually refresh the model with emerging slang and shifting sentiment cues.
- Cross-dataset validation with benchmark corpora (e.g., Ranjan et al. [18]) to assess portability across static and streaming contexts.

From an application standpoint, broadcasters and educators can follow audience engagement live; companies can track launch-day reactions; governments can sense public response to policy updates. Real-time studies capture sentiment in the moment, exposing sudden shifts and enabling immediate course-correction—benefits batch analyses cannot provide.

CONFLICT OF INTEREST

The authors declare that there are no conflicts of interest to declare with this publication.

COMPETING INTERESTS

The authors declare no competing interests.

DISCLAIMER

Portions of this manuscript were rephrased and refined for clarity using a large language model (ChatGPT, OpenAI). All technical content, analysis, results, and conclusions remain entirely the responsibility of the authors.

REFERENCES

[1] B. Liu, *Sentiment Analysis and Opinion Mining*. San Rafael, CA, USA: Morgan & Claypool, 2012.
[2] A. K. Joshi et al., "Harnessing social media for large-scale sentiment analysis," *Inf. Process. Manage.*, vol. 58, no. 3, pp. 102–116, 2021.
[3] H. Gao and J. Huang, "Real-time event analytics on social media streams," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 7, pp. 654–667, Jul. 2023.
[4] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.
[5] N. Marz and J. Warren, *Big data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island, NY, USA: Manning Publications, 2015.
[6] V. Gupta and R. Hewett, "Real-time tweet analytics using hybrid hashtags on Twitter big data streams," *Information*, vol. 11, no. 7, p. 341, Jun. 2020, doi: 10.3390/info11070341.
[7] R. K. Bakshi, N. Kaur, R. Kaur, and G. Kaur, "Opinion mining and sentiment analysis," *Found. Trends Inf. Retr.*, pp. 452–455, 2016.
[8] C. C. Aggarwal and C. Zhai, Eds., *Mining Text Data*. New York, NY, USA: Springer, 2012.
[9] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. EMNLP*, 2014, pp. 1746–1751.
[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
[11] Y. Zhao et al., "Overfitting risks in small-scale deep sentiment models," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 2, pp. 45–60, 2023.
[12] A. Kumar and P. Singh, "Twitter sentiment analysis using apache storm," *Proc. Comput. Sci.*, vol. 132, pp. 236–243, 2018.
[13] M. Ali, J. Han, and F. Hussain, "Real-time financial sentiment analysis with apache flink," *Future Gener. Comput. Syst.*, vol. 112, pp. 414–423, 2020.
[14] R. Gupta et al., "Comparative study of sentiment analysis on streaming vs. Batch data," *Future Gener. Comput. Syst.*, vol. 136, pp. 325–337, 2025.
[15] N. Patel, A. Dey, and R. Banerjee, "A kafka-spark streaming pipeline for IoT data processing," *J. Big Data Analytics Transp.*, vol. 7, no. 3, pp. 112–127, 2024.

[16] R. Arora et al., "Bias mitigation in multilingual sentiment analysis models," *IEEE Trans. Affect. Comput.*, vol. 16, no. 1, pp. 120–132, Jan. 2025.
[17] M. Jamil, H. Hadiyanto, and R. Sanjaya, "Sentiment analysis: Classifying public comments on YouTube in disaster management simulation in Indonesia using Naïve Bayes and support vector machine," *Ingénierie des Systèmes d Inf.*, vol. 29, no. 2, pp. 437–446, Apr. 2024.
[18] M. Ranjan, S. Tiwari, A. M. Sattar, and N. S. Tatkar, "A new approach for carrying out sentiment analysis of social media comments using natural language processing," *Eng. Proc.*, vol. 59, no. 1, 2024, Art. no. 181.
[19] S. S. Maw, E. C. Lwin, W. Mar, N. S. Paw, M. M. Khaing, and T. T. Aung, "Sentiment analysis with YouTube comments using deep learning approaches," in *Proc. IEEE Conf. Comput. Appl. (ICCA)*, Mar. 2024, pp. 1–7.
[20] R. Shinde, N. Patil, S. Nighute, R. Waghamare, and K. V. Metre, "Sentiment analysis of YouTube video," *Int. Res. J. Modernization Eng., Technol. Sci. (IRJMETS)*, vol. 6, no. 2, pp. 405–408, 2024.
[21] T. Padma, R. Visweshvar, K. Tamilarasan, and C. J. Bhadrinath, "Dynamic YouTube comment sentiment analysis with supervised fine-tuned BERT," in *Proc. Int. Conf. Cognit. Robot. Intell. Syst. (ICC-ROBINS)*, Apr. 2024, pp. 663–669.



SHETTY POORNIMA is an Assistant Professor-Senior Scale with the Directorate of Education (DOE), MAHE, Manipal, with extensive experience in university lecturing and higher education teaching. She has previously held academic positions with Mahatma Gandhi Memorial College, Alva's College, Poornaprajna College, and Sri Bhuvanendra College, contributing to higher education for over 18 years. Her research work has been recognized through publications in reputed journals and presentations at international conferences. With a strong commitment to academia, she continues to contribute to education and research in her field.



KINI SHASHIDHAR is a Professor and the Head of the Department of MCA, Srinivas Institute of Technology, Mangalore. With extensive expertise in Database Management, Data Mining, Management Information Systems, Unix Operating Systems, Software Engineering, and Data Modeling, he has a strong academic and research background. He has a proven track record in teaching, leadership, and team building while also guiding Ph.D. scholars in their research. His work has been published in IEEE, Springer, and other reputed journals. His research contributions have been presented in China, Singapore, and Chennai, further demonstrating his commitment to advancing knowledge in his field.

Dr. Kini is a Senior Member of the Institute of Research Engineers and Doctors (IRED), USA.



ROSHAN FERNANDES is a Professor and the Head of the Department of Cyber Security with NMAM Institute of Technology, NITTE, with over 21 years of experience in academia. He has previously served as an Associate Professor with the Department of Computer Science and Engineering for nearly two decades. He has actively contributed to the field of cybersecurity and computer science through his research and publications. Committed to academic excellence, he continues to mentor students and advance research in cybersecurity. His research contributions have been widely recognized, with 418 citations, an h-index of nine, and an i10-index of eight.